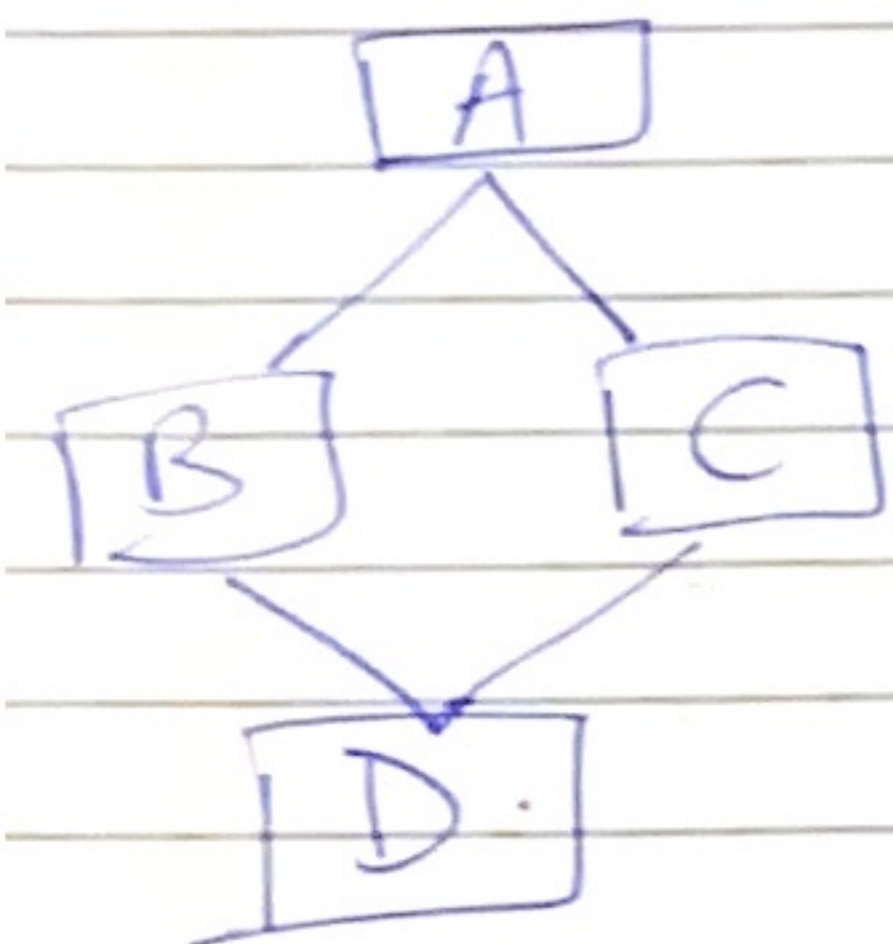


Not in Java.

Multiple Inheritance ~~(Diamond)~~:



```
class A
class B : public A
class C : public A
class D : public A, public B.
```

Issue: If B and C have print func. & 'D' does not have print func., the `D.print()` will cause error since compiler dk which class (B or C) print to call.

Solution: Use scope resolution operator. e.g: `B::print()`

Diamond Problem

Issue: Making a 'D' object will ultimately invoke constructor of A two times (`D → B → A` and `D → C → A`).

Solution: `class B: public virtual public A`
`class C: virtual public A`
`class D: public B, public C...`

Now constructor call from B & C will be skipped.

⇒ Pure Virtual: func. with no definition will be pure virtual if we make it virtual.

`virtual void salary() = 0;`

e.g: No. need for definition of `walk()` in Animal class. Define `walk()` func. in derived classes.

* A class with all virtual func. is "Interface" class.

* A class with at least one virtual func. is "Abstract" class.

~~POLYMORPHISM:~~

~~`void main() {
Animal * animals[5];`~~

`class calculator {`

`public:`

`void add(int a, int b) {`

`cout << a+b;`

`}`

`void add(float a, float b, float c) {`

`cout << a+b+c;`

`}`

⇒ GIST OF POLYMORPHISM:

- Compile time: Function overloading & operator overloading
- Run time: Virtual functions in (pointer) inheritance

→ **FUNCTION overload:** Can be used for string concatenate too.

```
class calculator {
public:
    void add (int a, int b) {...}
    void add (float a, float b, float c) {...}
}

int main() {
    calculator c;
    c.add(2,3);
    c.add(4.1, 3.2, 5);
}
```

→ **GIST OF POLYMORPHISM:** ~~overloading~~ loading

Compile time polymorphism: Function & operator overloading.

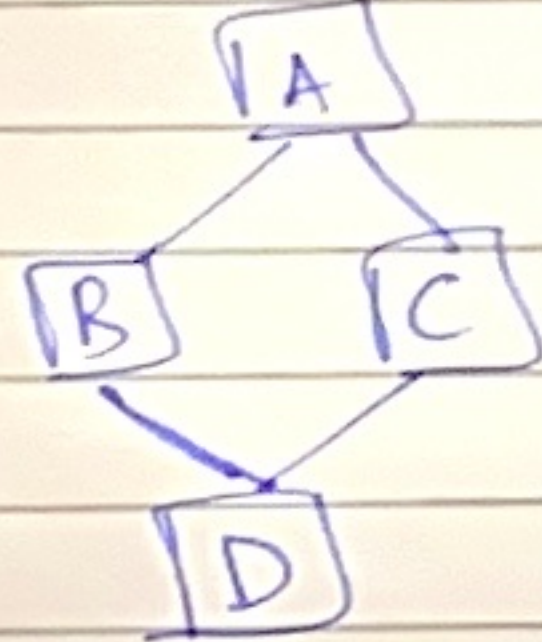
Runtime Polymorphism: Virtual function in Inheritance (pointer inheritance) or func overriding

→ **MULTIPLE INHERITANCE:**

```
class A {};
class B {};
class C: public A, public B {};
```

• An object of 'C' class can access all func. in A, B, C

→ **Diamond Problem:** (Not in Java) : Occurs when



```
class A {
public:
    void show() {....}
};
```

```
class B: public A {};
class C: public A {};
class D: public B, public C {};
```

```
int main() {
```

D obj;
 // "obj.show()" gives error since 'D' tries to access the show() in class A from two paths (D → B → A & D → C → A) so compiler gets confused. Constructor of A called two times too.

★ Every func. in base class isn't pure virtual instead of virtual. Since if we have Shape (base) class with virtual func. of display(), along with a circle class of over-written display() func. and a triangle class with no display so ~~Shape *s = new Triangle();~~ s.display(); will print shape's display without 'virtual inheritance'.

Solution: Virtual Inheritance

```
class A {
public:
    void show() {...}
};
class B: virtual public A {};
class C: virtual public A {};
class D: public B, public C {};
```

Solution: without 'virtual inheritance'.

```
int main() {
    D obj;
    obj.B::show();
    obj.C::show();
}
```

→ **Pure Virtual Function:** A func. with no definition in the base class and it must be over-ridden (defined) in the derived class(es).

- The class with at least one pure virtual func. is called "Abstract Class", and you CANNOT CREATE ITS OBJECT.
- Therefore if the pure virtual func. isn't over ridden (defined) in any ~~base~~ derived class, you cannot make object of that derived class as well.
- A class with only pure virtual func. is called "Interface Class".

```
class Animal {
public:
    virtual void speak() = 0;
}
```

```
class Dog: public Animal {
public:
    void speak() override {
        cout << "meow";
    }
}
```

```
int main() {
    Animal * a = new Dog();
    a → speak(); // meow
    // Animal a; X
    // Animal * a = new Animal(); X
}
```

OR

```
Animal * a
Dog d;
a = &d;
a → speak();
```


Single Class or Function could be used with different datatypes.
 To do operation which is independent of datatype

TEMPLATES: use "class" instead of 'typename' in class templates

template <typename T>

```
class MyList {
    T* myArray;
    int capacity;
    int currentIndex;
```

public:

```
MyList() {
    capacity = 2;
    myArray = new T[capacity];
    currentIndex = 0;
}
```

```
void AddElement(T element) {
    // adds element & doubles
    // the size of array if full
}
```

This will make the MyList & dog class interrel.
 The object 'd1' of type Dog class is passed in func. of MyList (i.e. AddElement).
 Since the ~~type~~ of template is 'Dog'; the template of MyList will create ~~an~~ array (named: myArray) of 'Dog' type and will add 'd1' in it.

⇒ More than 1 template:

```
template <typename T, typename Meow>
class MyList {
    T* myArr;
    Meow* values;
public:
    void Add(T key, Meow value) {}
}
```

⇒ Class Template:

```
int main() {
    MyList<int> meow;
    meow.AddElement(10);
    MyList<string> cat;
    cat.AddElement("hello");
    cat.Display();
}
```

⇒ Object Based Template:

```
class Dog {
    int age;
public:
    Dog(int a): age(a) {}
}
```

```
int main() {
    MyList<Dog> dogList;
    Dog d1(5); // d1 with age: 5;
    dogList.AddElement(d1);
}
```

```
int main() {
    MyList<int, string> obj;
    obj.Add(3, "Nice");
}
```

* class template: <class T>

* Function template: <typename T>

⇒ Function Templates:

```
template <typename T1, typename T2>
auto Sum(T1 data, T2 value) {
    return data + value;
}
```

→ chooses the return type itself. if we do int + float; the return type will get float.

```
template <typename T>
T Sum(T a, T b) {
    return a + b;
}
```

```
int main() {
    Sum(3, 5); // No need for
    Sum(4.5, 9.5); multiple
    Sum(5, 9.7); func. now
} where we've to differ parameters
for compile to understand which
'sum' are we calling.
```

⇒ Non-Type Templates:

```
template <typename T, int size>
class FixedArray {
    T arr[size];
};
```

for static array, this needs to be decided first, ∴ it is in the <>, so we already write it down in 'int main()' before running.

```
int main() {
    FixedArray<int, 5> meow;
    for (int i=0; i<5; i++) {
        meow.set(i); // makes an
    } // int array of size 5
```


func. overloading > func. specialisation } ^{overload} ↑ covers more cases

⇒ Template Specialization: If we've declared template class or func. already and we need ^{that} ~~class~~ class/func. to behave differently for any one specific datatype, we'll use template specialization.

→ Class:

```
template <typename T>
class Spunky {
public:
    void Print (T value) {
        cout << value << "is not a character";
    }
}
```

} Simple template for all datatypes

```
template <>
class Spunky <char>
public:
    void Print (char value) {
        cout << value << "is indeed character";
    }
}
```

} Specialized template for "char" datatype.

```
int main () {
    Spunky <int> obj 1;
    Spunky <float> obj 2;
    Spunky <char> obj 3;
    obj 1. Print (3);
    obj 2. Print (4.13);
    obj 3. Print ('K');
```

Output:
 3 is not a character
 4.13 is not a character
 K is indeed a character.

→ Func. (template specialization).

```
template <typename T>
T show (T value) {
    return value;
}

template <>
int show <int> (int value) {
    return value + 2;
}
```

```
template <typename T>
T maximum (T x, T y) {
    if (x > y)
        return x;
    else
        return y;
}
```

```
void main () {
    cout << maximum (55, 88);
    cout << maximum ('A', 'u');
    char arr [5] = "sdsc";
    char arr2 [4] = "mow";
    cout << maximum (arr, arr2);
}
```

```
template <>
char * maximum <char*> (char* x, char* y) {
    if (strcmp (x, y) == 1)
        return x;
    else
        return y;
}
```

uses simple template (defined above)

• uses specialised template for char array
 • Need to specialised too after 'const char*' ho.

UPCASTING: A parent class pointer pointing a memory of child class.

Animal * a = new dog(); OR Dog d; Animal * a = &d;

* But we cannot point a parent class memory by a child class pointer directly.

Dog * d = new Animal(); X

↓
 For this, downcasting is used.

object.

□ DOWNCASTING: To point a parent class memory by child class.

/object slicing

① C-Style downcasting: Unsafe as no runtime checks to see if 'a' pointer is actually pointing a dog memory or not. Leads to UNDEFINED BEHAVIOUR.

```
Animal * a = new Dog();
Dog * d = (Dog *) a; // style cast
d->Speak();
delete a;
```

This will force compiler to think that pointer 'a' is pointing a Dog() memory even if it's not. Even if we won't use virtual with speak() func. of parent class (Animal); it wld still output Dog's speak() func. since pointer 'd' is pointing a dog() type memory due to "C-style cast".

```
Animal * a = new Animal();
Dog * d = (Dog *) a;
d->Speak();
delete a;
```

Since 'a' isn't pointing a dog() type memory so undefined behaviour.

② Static Downcasting: Almost same as C-style downcasting but a little more safe. No checks of whether the pointer 'a' actually points dog() memory or not so undefined behaviour.

```
Animal * a = new Animal();
Dog * d = static_cast<Dog*>(a);
d->Speak();
delete a;
```

makes the compiler assume that 'a' pointer points this Dog() memory so it'll assign that memory to 'd'.

③ Dynamic Cast Downcasting: Safe since runtime checks are run to ensure that object is of correct type before casting (means 'a' is pointing correct dog type memory).

```
Animal * a = new Dog();
Dog * d = dynamic_cast<Dog*>(a);
if (d) {
    d->Speak(); // wld only work if 'a' is actually pointing
                // a dog type memory (like above).
} else {
    cout << "Invalid cast"; // if "Animal * a = new Animal();"
}
delete a;
```

□ Final Keyword:

⇒ Class:
class A final {
};
No class wld be able to inherit class 'A' now.

⇒ Function:
class A {
 virtual void print() {...}
};
class B: public A {
 void print() override final {...}
}
class C: public B {
 void print() override {
 }

→ This would give error since 'C' class is inheriting 'B' class but 'B' class has put a full stop to further override print() func. by writing 'final'. So any class inheriting 'B' won't be able to override print().